



UFR STA Licence 2 MPI

Système d'Exploitation (SE)

Dr Lamine YADE
lameyade9@gmail.com

Année académique: 2024-2025

Objectif:

Ce cours a pour objet:

- de décrire le **rôle** et le **fonctionnement** d'un système d'exploitation
- d'apprendre à manipuler certains **concepts de base**.
- Ces concepts de base seront essentiellement illustrés au travers du système **Windows** & **UNIX/Linux**

Les étudiants doivent connaître:

- la **définition** d'un SE
- le **rôle** d'un SE
- les **classes** de SE
- le **mécanisme de base** d'un SE : processus, Entrées/Sorties, etc.
- le **Système de Gestion des Fichiers** SGF

PLAN

I. Généralité sur les Systèmes d'Exploitation

- Fonctions de base d'un SE
 - Gestion des processus
 - Gestion de la mémoire
 - Les Entrées/Sorties
- Différents types d'architecture de SE

II. Système Gestion des Fichier (SGF)

- Sous Windows
- Sous Linux

III. Linux utilisateur

IV. Travaux Pratiques (TP)

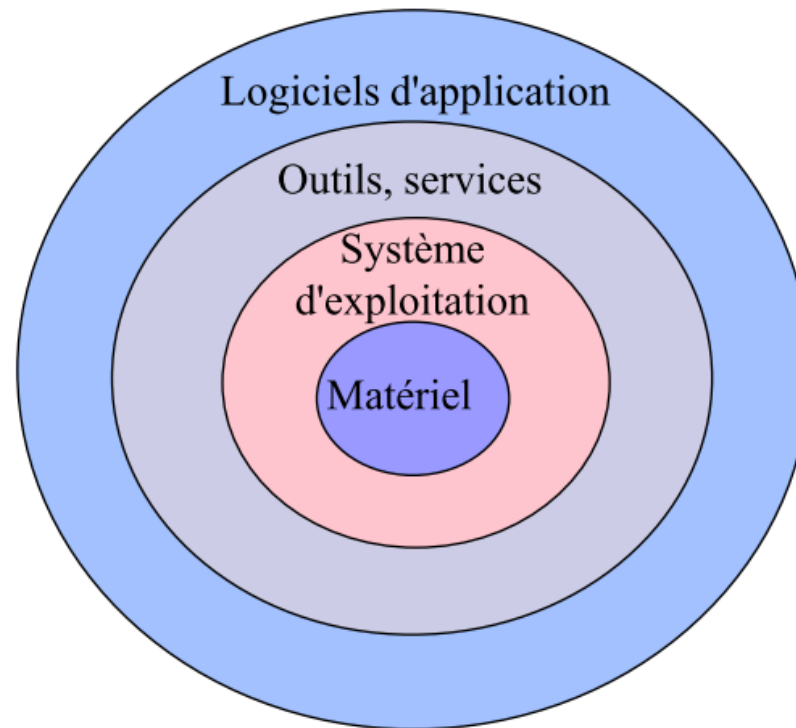
I. Généralité sur les SE

I.1. Système Informatique (SI)

- ❖ Le mot **informatique**(**INFOR**mation auto**MATIQUE**), proposé en **1962** par **Philippe Dreyfus** du centre de calcul de **Bull**, définit la science de *traitement automatique de l'information par l'ordinateur*.
- ❖ Un **Ordinateur** = Machine électronique **programmable** capable de réaliser des **calculs logiques** sur des nombres **binaires**.

I.1. Système Informatique (SI)

- ❖ Un **SI** est donc l'ensemble de moyens **matériels** et **logiciels** destinés à réaliser des tâches qui mettent en jeu le traitement automatique de l'information dans le but de satisfaire les besoins informatiques des utilisateurs



$$\mathbf{SI} = \mathbf{Matériels} + \mathbf{Logiciels}$$

I.1. Système Informatique: Fonctions de base

Les fonctions de base d'un SI:

Communication avec l'utilisateur

- **entrée** : entrée des information à traiter : les **données**
- **sortie** : communication des **résultats**

Calcul (ou traitement):

- élaboration des **résultats** à partir des **données**

Séquencement:

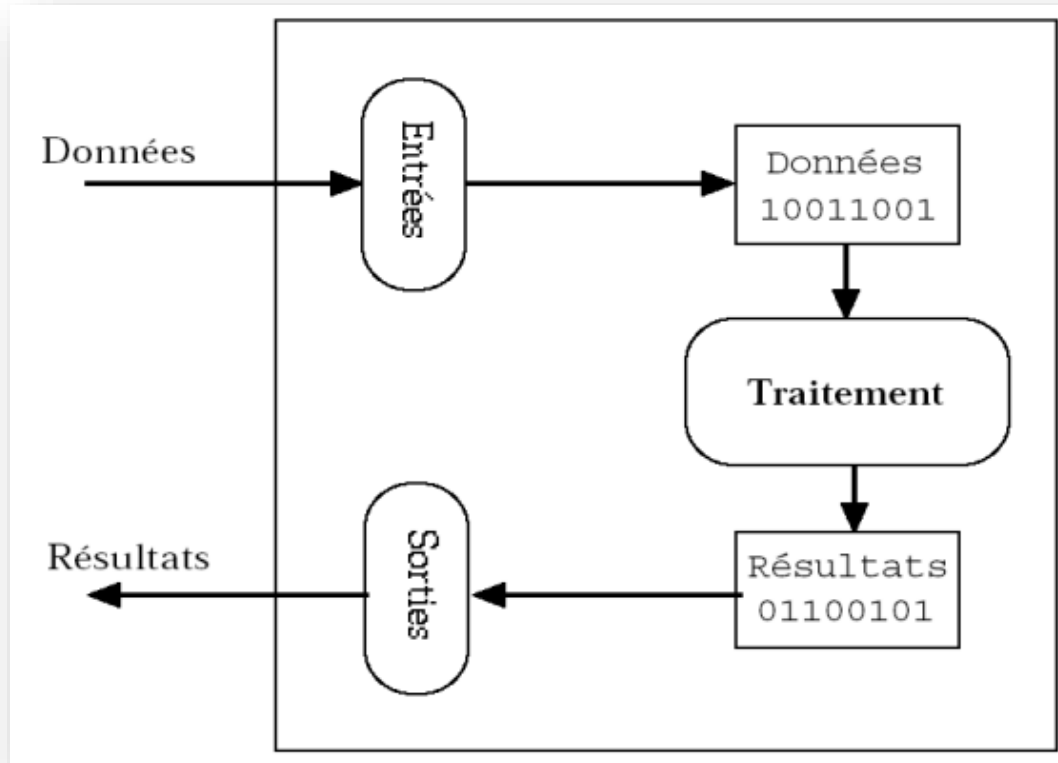
- Les opérations sont exécutées l'une après l'autre selon un certain plan appelé un **programme**.
- Le choix d'une opération peut éventuellement dépendre du résultat des opérations précédentes

Mémorisation:

- Les **données** et les **résultats** doivent pouvoir être **mémorisés**
- La séquence des opérations à effectuer (programme) est également mémorisée

I.1. Système Informatique (SI): Fonctions de base

Exploitation d'un système informatique



I.2. Partie matériel (Hardware)

Le matériel (Hardware): Les dispositifs physiques constituent la couche la plus basse du matériel.

Matériel (Hardware) = Unité centrale + Périphériques

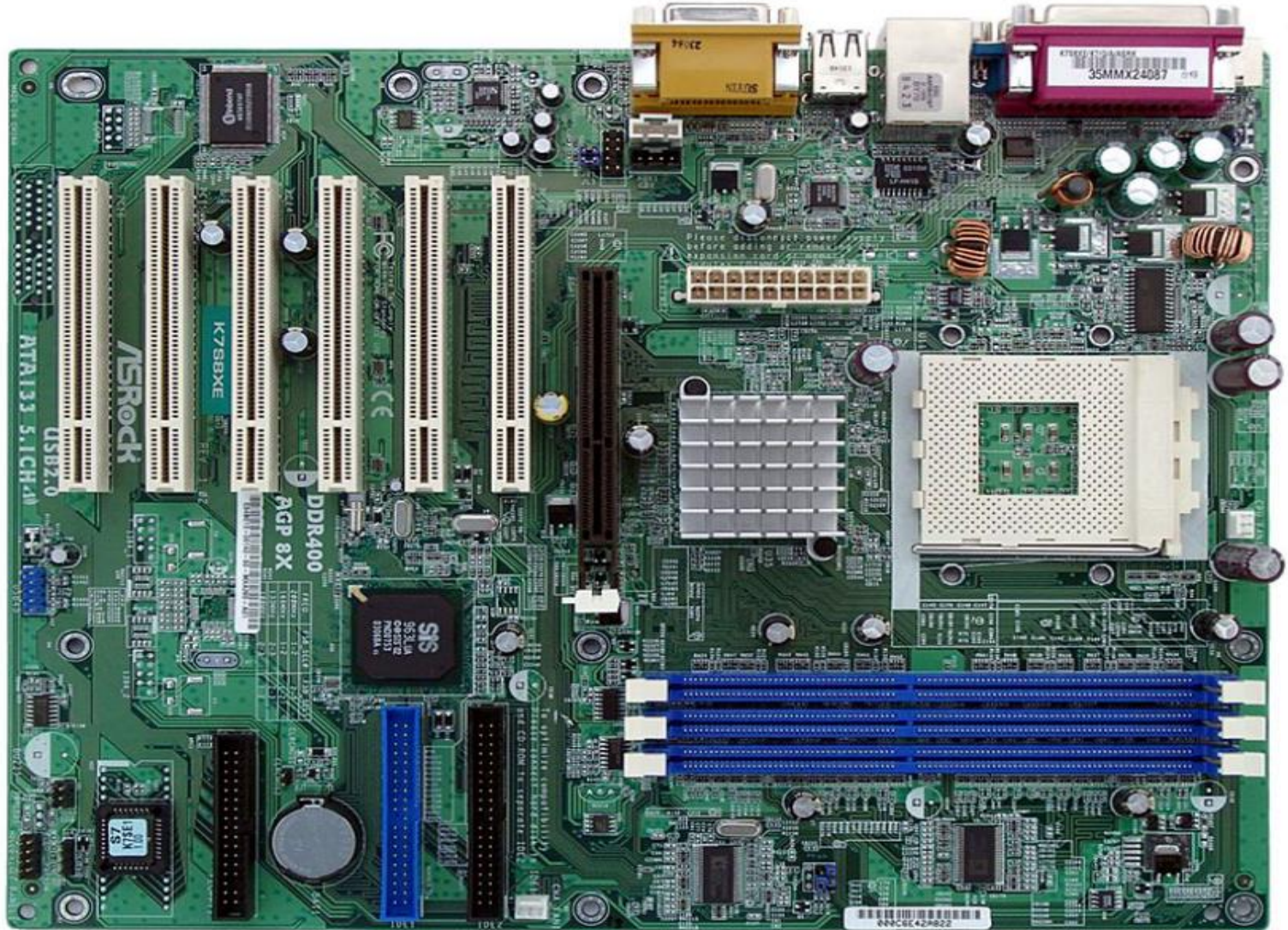
Unité centrale: c'est une boîte qui comporte:

- **La carte mère:** Support de connections reliant les matériels
- **Le processeur:** unité de traitement
- **La mémoire centrale :** RAM ou mémoire volatile
- **Les mémoires de masse:** ROM ou supports permanents
- **Etc.**

Périphériques: les matériels connectés à l'**unité centrale** et qui servent à

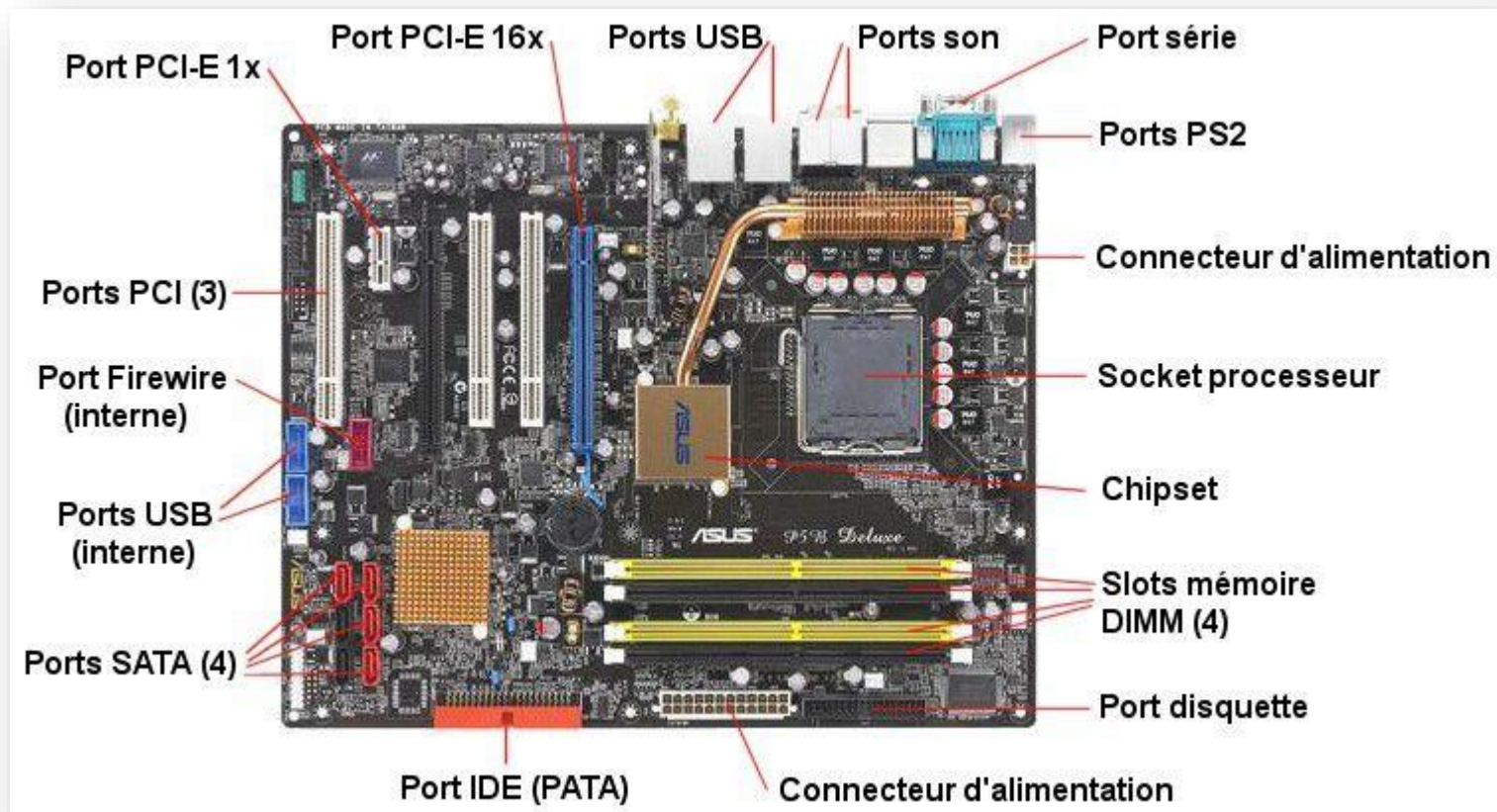
- Entrer
- sortir des informations

I.2. Partie matériel (Hardware): Carte mère

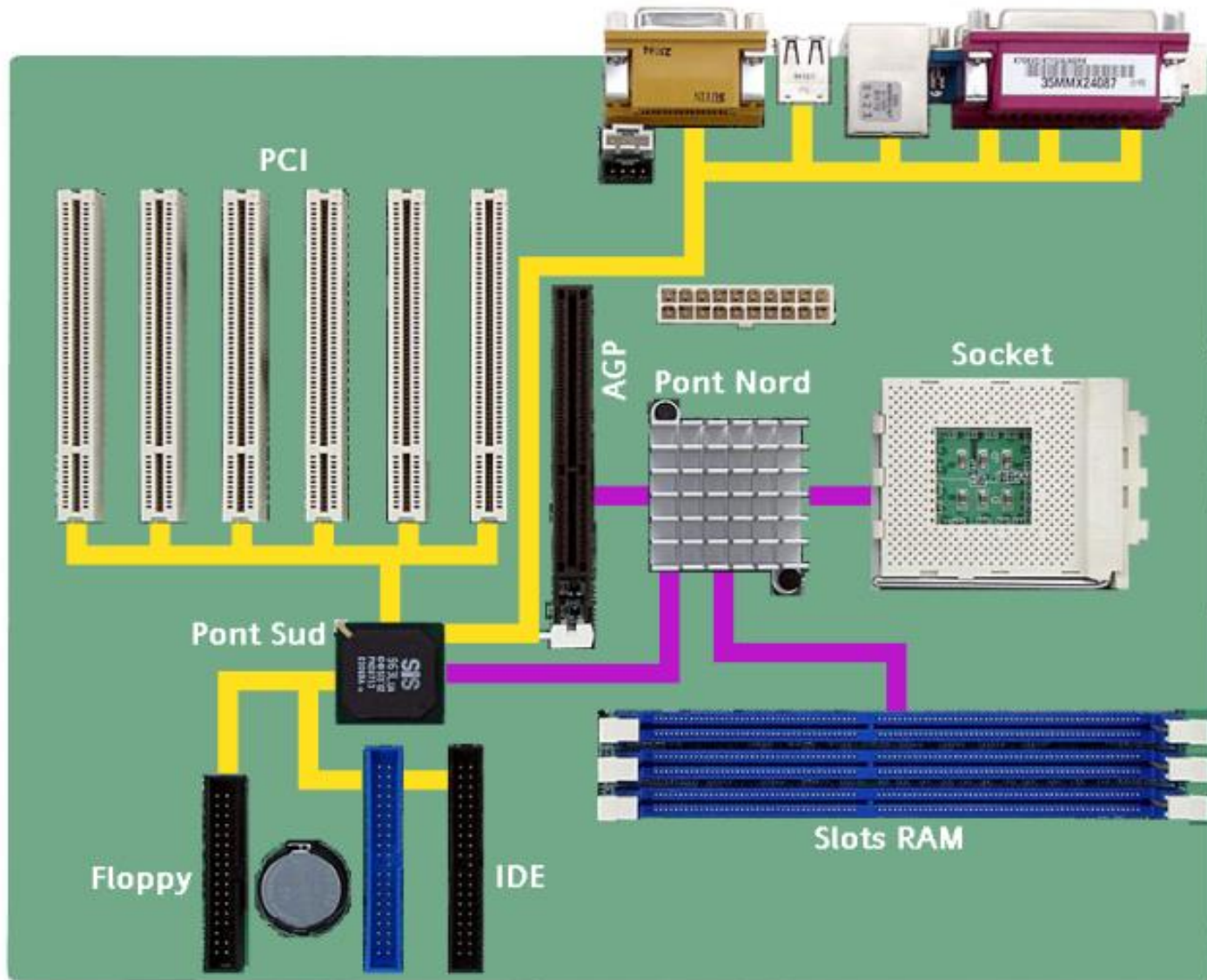


I.2. Partie matériel (Hardware)

Carte mere: c'est un matériel informatique composé de circuits imprimés et de ports de connexion et qui servant à interconnecter toutes les composantes d'un ordinateur par l'intermédiaire du **Bus**.



I.2. Partie matériel (Hardware): Bus



I.2. Partie matériel (Hardware)

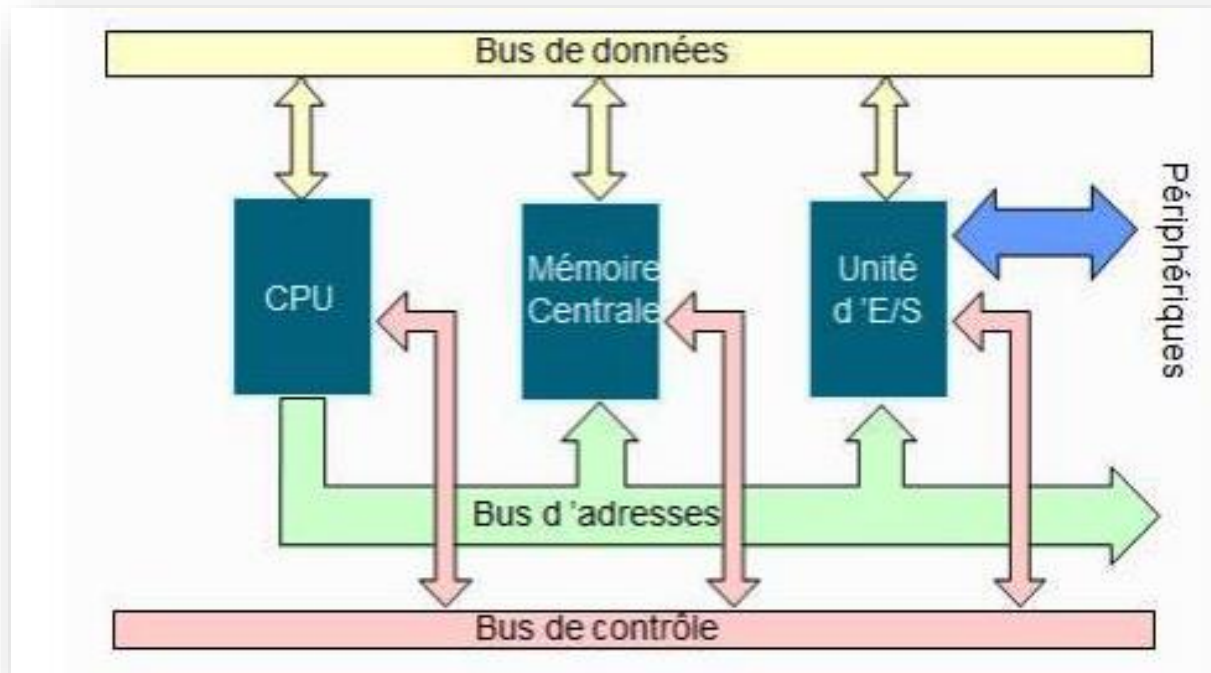
Bus (ensemble de lignes électriques): est un circuit intégré à la **carte-mère** qui assure la **circulation des données** entre les différents éléments du PC (mémoire vive, carte graphique, USB, etc.)

On distingue généralement deux types de **bus**:

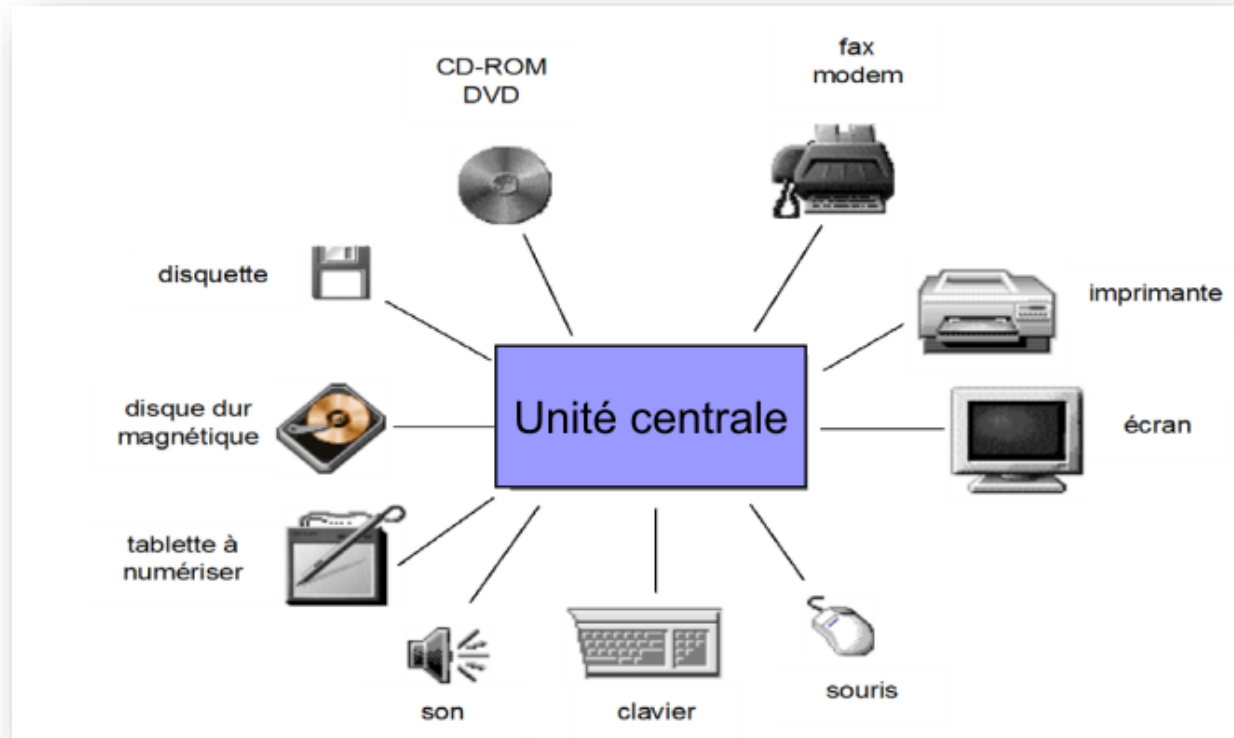
- **Le bus système** (appelé aussi bus interne, en anglais internal bus ou front-side bus, noté FSB) : il relie le **microprocesseur** à la **mémoire vive**
- **Les bus d'extension** (bus ISA, bus AGP et bus PCI) ou bus d'entrées/sorties : ils relient le **microprocesseur** aux connecteurs d'entrées/sorties et aux connecteurs d'extension.

I.2. Partie matériel (Hardware)

Bus (ensemble de lignes électriques): est un circuit intégré à la **carte-mère** qui assure la **circulation des données** entre les différents éléments du PC (mémoire vive, carte graphique, USB, etc.)



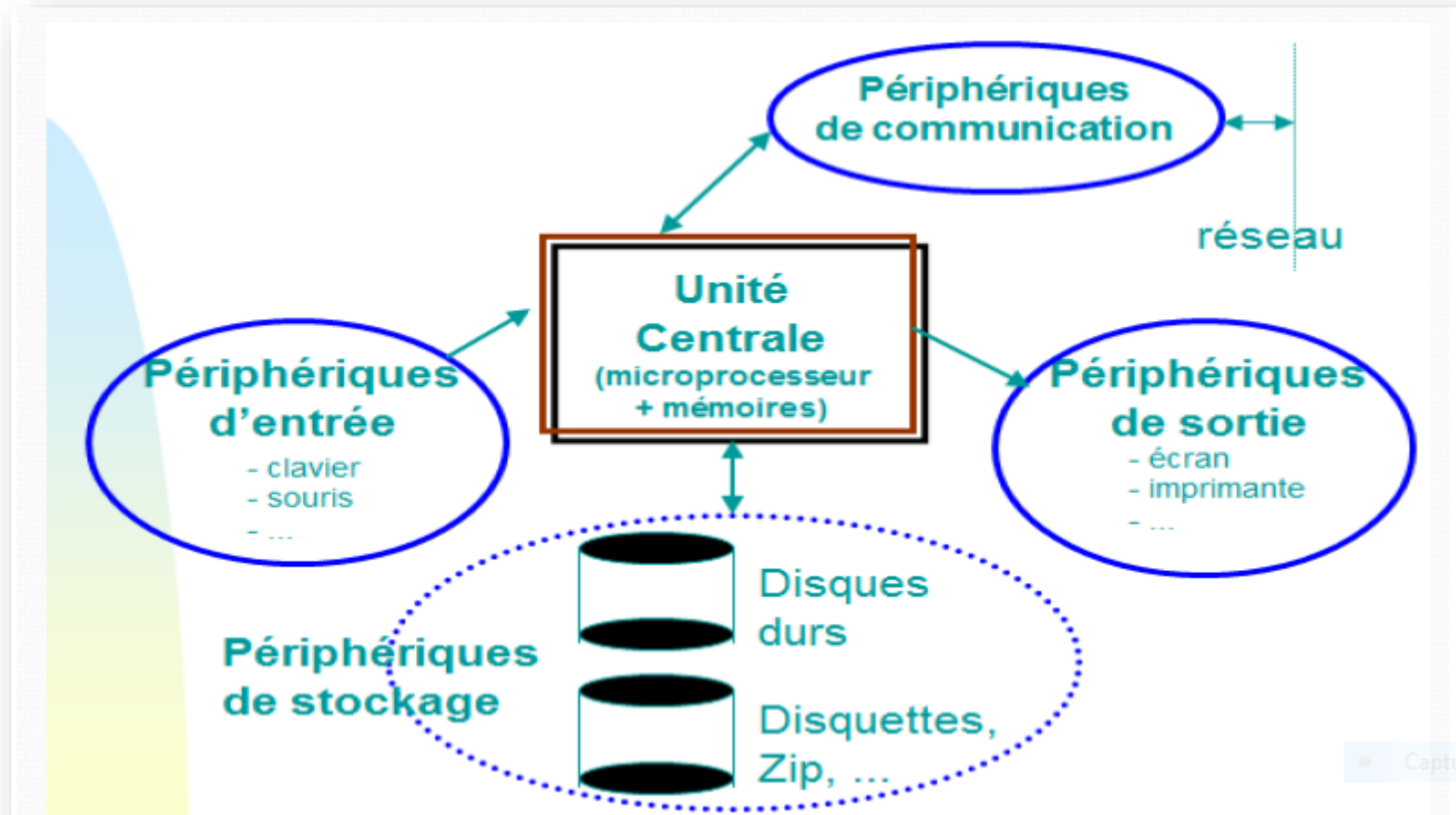
I.2. Partie matériel (Hardware)



Matériel (Hardware) = Unité centrale + Périphériques

I.2. La partie matériel (Hardware)

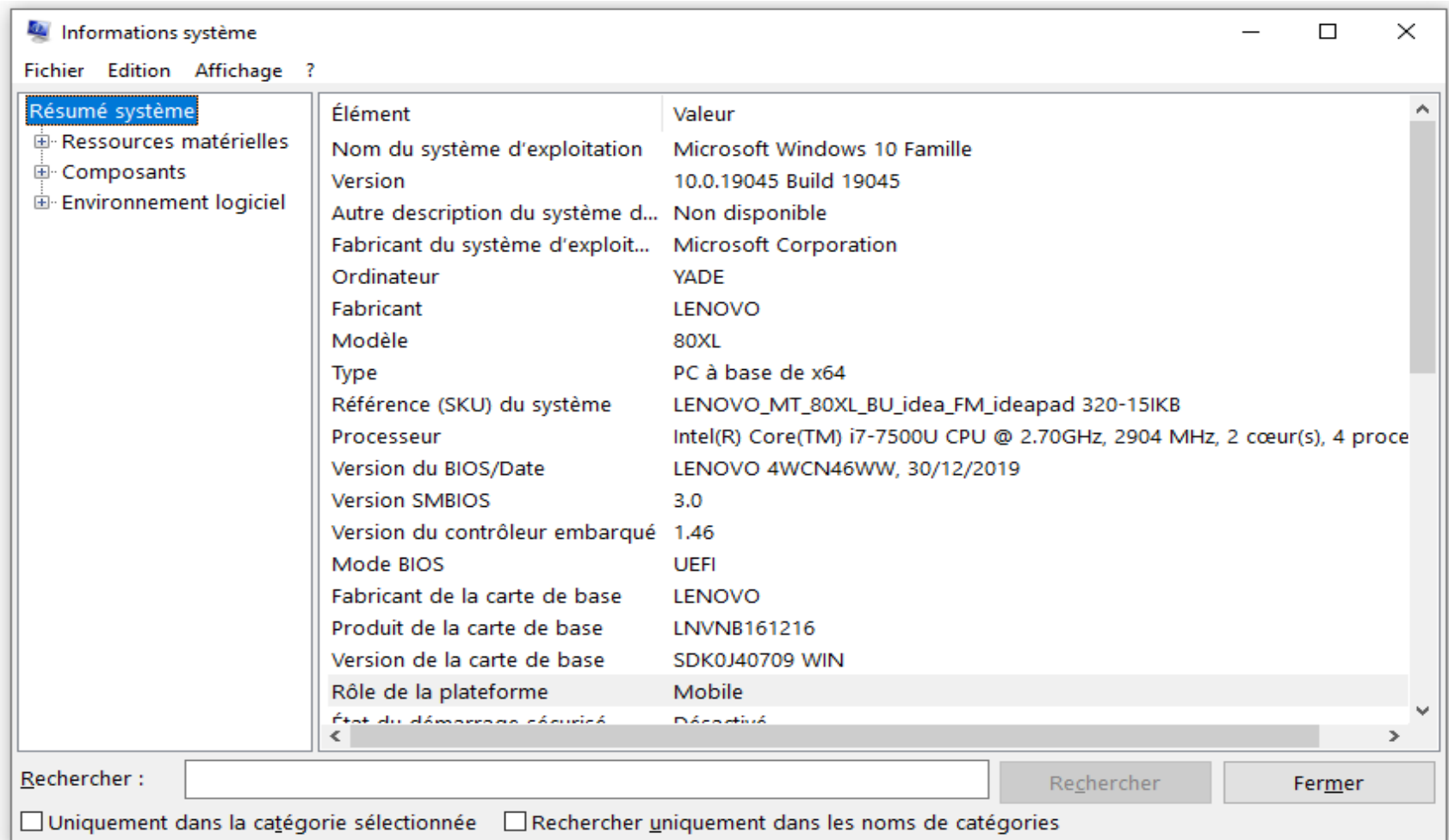
Schema materiel



Matériel (Hardware) = Unité centrale + Périphériques

I.2. La partie matériel (Hardware)

❖ Afficher informations : Commande = **msinfo32** ou **systeminfo**



Élément	Valeur
Nom du système d'exploitation	Microsoft Windows 10 Famille
Version	10.0.19045 Build 19045
Autre description du système d'exploitation	Non disponible
Fabricant du système d'exploitation	Microsoft Corporation
Ordinateur	YADE
Fabricant	LENOVO
Modèle	80XL
Type	PC à base de x64
Référence (SKU) du système	LENOVO_MT_80XL_BU_idea_FM_ideapad 320-15IKB
Processeur	Intel(R) Core(TM) i7-7500U CPU @ 2.70GHz, 2904 MHz, 2 cœur(s), 4 processeur(s)
Version du BIOS/Date	LENOVO 4WCN46WW, 30/12/2019
Version SMBIOS	3.0
Version du contrôleur embarqué	1.46
Mode BIOS	UEFI
Fabricant de la carte de base	LENOVO
Produit de la carte de base	LNVNB161216
Version de la carte de base	SDK0J40709 WIN
Rôle de la plateforme	Mobile
État du démarrage sécurisé	Désactivé

Rechercher :

☐ Uniquement dans la catégorie sélectionnée ☐ Rechercher uniquement dans les noms de catégories

I.2. La partie matériel (Hardware)

❖ **Afficher informations :** Commande = **wmic**

Exemples d'utilisation:

Afficher la taille et le nom des lecteurs de disque: *wmic diskdrive get size,name*

```
wmic:root\cli>diskdrive get size, name
Name                               Size
\\.\PHYSICALDRIVE0                500105249280

wmic:root\cli>
```

Afficher informations de la carte mère: *wmic baseboard get product,manufacturer,version*

```
C:\Users\Administrateur>wmic
wmic:root\cli>baseboard get product,manufacturer,version
Manufacturer  Product      Version
LENOVO         LNVNB161216  SDK0J40709 WIN

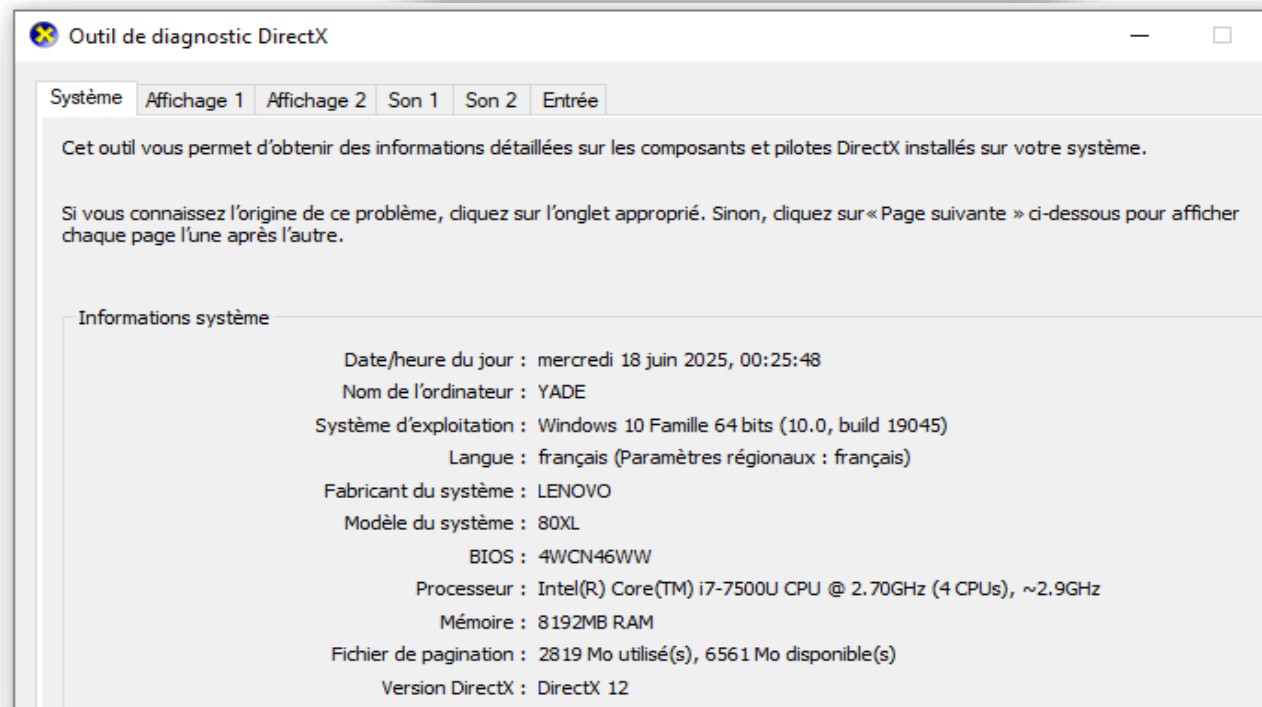
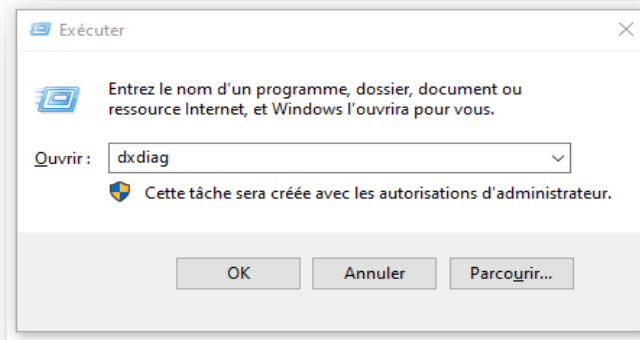
wmic:root\cli>
```

I.2. La partie matériel (Hardware)

❖ Diagnostique



+ R : **dxdiag**



I.3. Partie logicielle(Software)

- ❖ Un système informatique est composé de deux niveaux bien distincts : le **système d'exploitation** et les **applications**.
 - L'**objectif** du logiciel est d'**offrir aux utilisateurs des fonctionnalités** adaptées à leurs besoins.
 - Le **principe** est de **masquer les caractéristiques physiques** du **matériel**.
 - La **solution** consiste à **structurer** en couches le **logiciel**, chacune offrant des services de plus en plus évolués au niveau supérieur.

I.3. Partie logicielle(Software)

- ❖ Un **logiciel** est un **programme** qui décrit une procédure de calcul qui à partir d'informations en **entrée** (données) produit des informations en **sortie** (résultats).
- ❖ Deux types de logiciel existent:
 - **Logiciel de base** : Le **Système d'Exploitation** et les logiciels de développement
 - **Logiciel d'application(Programmes)**

Partie logicielle(Software) = Système d'Exploitation + Programmes

Programmes :

- Programmes système
- Programmes d'applications

I.3.1 Le Système d'Exploitation (SE)

❖ Définition:

- Le **Système d'Exploitation(SE)** ou **Operating System (OS)** : est un logiciel destiné à faire fonctionner l'ordinateur.
- Il sert **d'intermédiaire** entre **l'utilisateur** d'un ordinateur et le **matériel**

Il se compose:

- **D'un noyau (kernel):** partie la plus critique d'un **OS**. Il permet aux éléments **matériel** et **logiciel** de communiquer entre eux, de fonctionner ensemble et de former un tout. Pour ces raisons, il est le premier logiciel chargé en mémoire.
- **Des outils système:** partie permettant à l'utilisateur de tirer profit de l'OS, de gérer les périphériques, les configurer ...En bref, ils fournissent une interface d'accès au système.

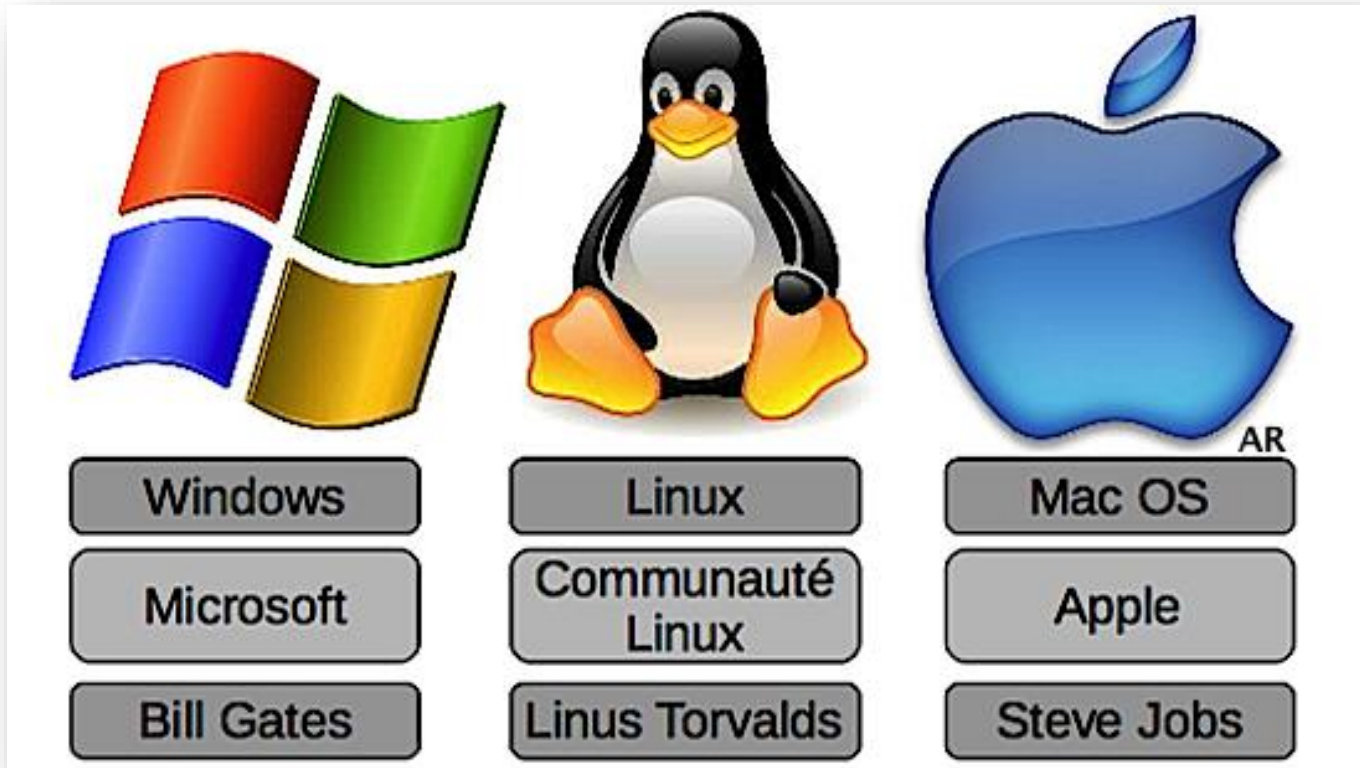
I.3.1 Le Système d'Exploitation (SE)

❖ Exemple de SE



I.3.1 Le Système d'Exploitation (SE)

❖ Exemple de SE



I.3.1 Le Système d'Exploitation (SE)

❖ Exemple de SE pour smartphones

- **Android:** Développé par Google
- **iOS:** Développé par Apple pour iPhones, iPads
- **Bada:** Samsung a finalement abandonné en 2013 au profit **Tizen**
- **BlackBerry OS (Recherche en mouvement)**
- **Windows:** Microsoft
- **Symbian :** Nokia
- **Tizen :** Samsung
- **Android TV:** Google, pour les smart TV et les appareils de streaming
- **FreeBSD**
- **Etc.**

1.3.1 Le Système d'Exploitation (SE)

❖ Exemple de SE



1.3.1 Le Système d'Exploitation (SE)

Tâches:

- ✓ Assure le démarrage de l'ordinateur
- ✓ Fournir aux programmes applicatifs des interfaces standardisées pour l'accès aux périphériques.
- ✓ Fournir à l'utilisateur une machine étendue ou virtuelle, plus simple à programmer
- ✓ Contrôler les ressources de l'ordinateur et libérer le programmeur de la complexité du matériel. Deux dimensions du partage(multiplexage) :
 - Temps
 - espace

1.3.1 Le Système d'Exploitation (SE)

Fonctions du SE:

- ❖ **gestion de l'information** : structuration, désignation de l'information, stockage, recherche, communication. Fonction réalisée par le **Système de Gestion de Fichiers (SGF)**.
- ❖ **Fonction d'exécution** : exécution de programmes (applications) en séquence, en parallèle, composition de programmes. Fonction réalisée par **l'interprète du langage de commande**.
- ❖ **Gestion des ressources physiques** :
 - allocation de la mémoire principale,
 - allocation de la mémoire secondaire,
 - allocation des organes d'entrée-sortie,
 - allocation du processeur

1.3.1 Le Système d'Exploitation (SE)

Fonctions du SE:

- ❖ **Partage et échange d'informations entre usagers :** : messagerie, liens symboliques, gestion des droits d'accès.

- ❖ **Services divers :**
 - aide à la mise au point de programmes (débugueurs),
 - traitement des défaillances,
 - mesure du temps,
 - facturation des ressources,
 - statistiques d'utilisation des ressources,
 - mesure de performances,
 - etc.

1.3.1 Le Système d'Exploitation (SE)

❖ **Afficher informations : Commande = wmic**

Afficher le nom et la version du système d'exploitation:

os get caption,version

```
C:\Users\Administrateur>wmic
wmic:root\cli>os get caption,version
Caption                                Version
Microsoft Windows 10 Famille         10.0.19045
```

1.3.1 Le Système d'Exploitation (SE)

❖ Afficher informations :

winver



systeminfo

C:\Users\Administrateur>systeminfo

Nom de l'hôte:	YADE
Nom du système d'exploitation:	Microsoft Windows 10 Famille
Version du système:	10.0.19045 N/A build 19045
Fabricant du système d'exploitation:	Microsoft Corporation
Configuration du système d'exploitation:	Station de travail autonome
Type de build du système d'exploitation:	Multiprocessor Free
Propriétaire enregistré:	moi
Organisation enregistrée:	
Identificateur de produit:	00325-96225-81125-AAOEM
Date d'installation originale:	26/07/2023, 18:30:47
Heure de démarrage du système:	18/06/2025, 00:23:21
Fabricant du système:	LENOVO
Modèle du système:	80XL
Type du système:	x64-based PC
Processeur(s):	1 processeur(s) installé(s). [01] : Intel64 Family 6 Model 142 Stepping
	9 GenuineIntel ~2703 MHz
Version du BIOS:	LENOVO 4WCN46WW, 30/12/2019

I.4. Fonctions de base d'un SE

- ❖ Les principales fonctions assurées par un SE sont les suivantes :
- gestion de la **mémoire** principale et des mémoires secondaires,
- exécution des **E/S** à faible débit (terminaux, imprimantes) ou haut débit (disques, bandes),
- **multiprogrammation**, temps partagé, parallélisme : interruption, ordonnancement, répartition en mémoire, partage des données,
- lancement des outils du système (compilateurs, environnement utilisateur,...) et des outils pour l'administrateur du système (création de points d'entrée, modification de privilèges,...),
- protection, sécurité,
- réseaux

I.4. Fonctions de base d'un SE

- ❖ **L'interface entre un SE et les programmes utilisateurs** est constituée d'un ensemble d'instructions étendues, spécifiques d'un SE, ou appels système.
- ❖ Généralement, les appels système concernent soit les **processus(perfmon)**, soit le système de gestion de fichiers (**SGF**).

I.4.1. Gestion des processus

- ❖ Un **processus** est un **programme** qui **s'exécute**, ainsi que ses données, sa pile, son compteur ordinal, son pointeur de pile et les autres contenus de registres nécessaires à son exécution.
- ❖ Afficher les processus en cours d'exécution:

wmic process get name,executablepath ou tasklist (cmd)

```
C:\Windows\system32\svchost.exe
C:\Windows\system32\taskhostw.exe
C:\Windows\system32\svchost.exe
C:\Windows\System32\svchost.exe
C:\Windows\system32\ctfmon.exe
C:\Windows\Explorer.EXE
C:\Windows\system32\svchost.exe
C:\Windows\System32\DriverStore\FileRepository\cui_dch.inf_amd64_38cfab2b652e4701\igfx
C:\Windows\system32\svchost.exe
C:\Windows\SystemApps\Microsoft.Windows.StartMenuExperienceHost_cw5n1h2txyewy\StartMen
C:\Windows\System32\RuntimeBroker.exe
C:\Windows\system32\svchost.exe
C:\Windows\SystemApps\Microsoft.Windows.Search_cw5n1h2txyewy\SearchApp.exe
C:\Windows\System32\RuntimeBroker.exe
C:\Windows\system32\svchost.exe
C:\Windows\system32\svchost.exe
C:\Program Files\WindowsApps\Microsoft.YourPhone_1.25042.96.0_x64__8wekyb3d8bbwe\Phone
C:\Windows\System32\RuntimeBroker.exe
C:\Windows\system32\cmd.exe
C:\Windows\system32\conhost.exe
C:\Windows\System32\SecurityHealthSystray.exe
```

• Capture rectangulaire

I.4.1. Gestion des processus

- ❖ Les appels système relatifs aux **processus** permettent généralement d'effectuer au moins les actions suivantes :
 - **création** d'un processus (fils) par un processus actif (d'où la structure d'arbre de processus gérée par un SE)
 - **destruction**, mise en **attente**, **réveil** d'un processus
 - **suspension** et **reprise** d'un processus, grâce à l'**ordonnanceur** de processus (scheduler)
 - demande de mémoire supplémentaire ou restitution de mémoire inutilisée
 - attendre la fin d'un processus fils
 - remplacer son propre code par celui d'un programme différent
 - échanges de messages avec d'autres processus
 - spécification d'actions à entreprendre en fonction d'événements extérieurs asynchrones
 - modifier la priorité d'un processus

I.4.2. Les interruption

- ❖ Une **interruption** est une commutation du mot d'état provoquée par un **signal généré par le matériel**.
- ❖ Le **signal** va modifier la valeur d'un indicateur qui est consulté par le SE. Celui-ci est ainsi informé de l'arrivée de l'interruption et de son origine.
- ❖ A chaque cause d'interruption est associé un niveau d'interruption.
- ❖ On distingue au moins 3 niveaux d'interruption :
 - les **interruptions externes** : panne, intervention de l'opérateur,
 - les **déroutements** qui proviennent d'une situation exceptionnelle ou d'une erreur liée à l'instruction en cours d'exécution (division par 0, débordement, ...)
 - les **appels système**

UNIX admet 6 niveaux d'interruption : interruption **horloge**, interruption **disque**, interruption **console**, interruption d'un autre **périphérique**, **appel système**, autre interruption.

I.4.3. Les ressources

- ❖ On appelle **ressource** tout ce qui est nécessaire à l'avancement d'un **processus** (continuation ou progression de l'exécution) : processeur, mémoire, périphérique, bus, réseau, compilateur, fichier, message d'un autre processus, etc...
- ❖ Un **défaut de ressource peut provoquer la mise en attente d'un processus.**
- ❖ Un processus **demande** au SE l'**accès** à une ressource.
- ❖ Le SE **alloue** une ressource à un processus. Une fois une ressource allouée, le processus a le droit de l'utiliser jusqu'à ce qu'il **libère** la ressource ou jusqu'à ce que le **SE reprenne la ressource** (on parle en ce cas de ressource préemptible, de préemption).

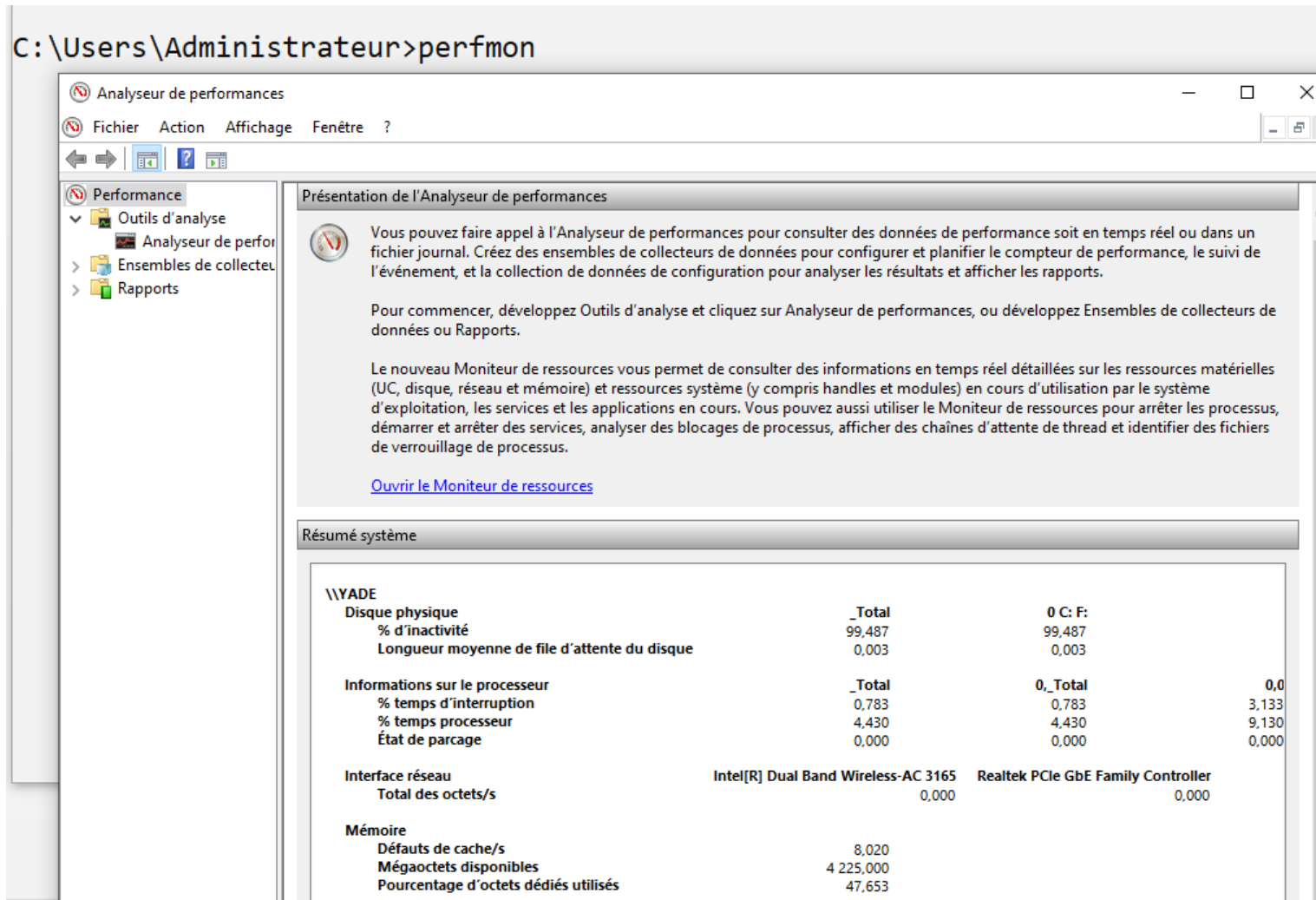
I.4.3. Les ressources

- ❖ On dit qu'une ressource est en mode d'**accès exclusif** si elle ne peut être allouée à plus d'un processus à la fois. Sinon, on parle de mode d'**accès partagé**.
- ❖ Le **mode d'accès** à une ressource dépend largement de ses **caractéristiques technologiques**.
- ❖ Deux ressources sont dites **équivalentes** si elles assurent les **mêmes fonctions** vis à vis du processus demandeur.

I.4.3. Les ressources

❖ Afficher: Commande **perfmon**

```
C:\Users\Administrateur>perfmon
```



Présentation de l'Analyseur de performances

Vous pouvez faire appel à l'Analyseur de performances pour consulter des données de performance soit en temps réel ou dans un fichier journal. Créez des ensembles de collecteurs de données pour configurer et planifier le compteur de performance, le suivi de l'événement, et la collection de données de configuration pour analyser les résultats et afficher les rapports.

Pour commencer, développez Outils d'analyse et cliquez sur Analyseur de performances, ou développez Ensembles de collecteurs de données ou Rapports.

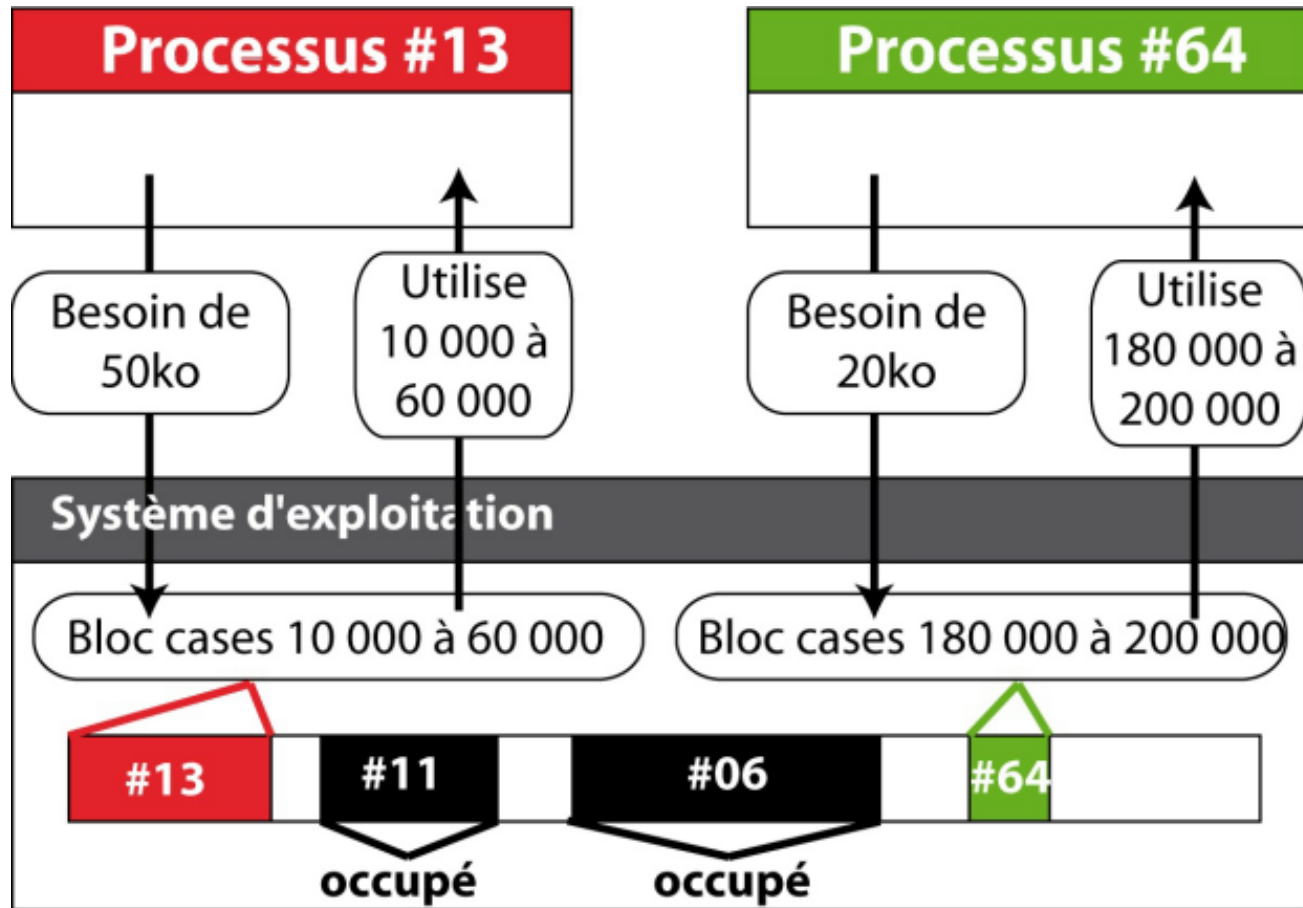
Le nouveau Moniteur de ressources vous permet de consulter des informations en temps réel détaillées sur les ressources matérielles (UC, disque, réseau et mémoire) et ressources système (y compris handles et modules) en cours d'utilisation par le système d'exploitation, les services et les applications en cours. Vous pouvez aussi utiliser le Moniteur de ressources pour arrêter les processus, démarrer et arrêter des services, analyser des blocages de processus, afficher des chaînes d'attente de thread et identifier des fichiers de verrouillage de processus.

[Ouvrir le Moniteur de ressources](#)

Résumé système

\\.\ADE			
Disque physique			
% d'inactivité	_Total	0 C: F:	
	99,487	99,487	
Longueur moyenne de file d'attente du disque	0,003	0,003	
Informations sur le processeur			
	_Total	0, _Total	0,0
% temps d'interruption	0,783	0,783	3,133
% temps processeur	4,430	4,430	9,130
État de parage	0,000	0,000	0,000
Interface réseau			
Total des octets/s	Intel[R] Dual Band Wireless-AC 3165	Realtek PCIe GbE Family Controller	
	0,000	0,000	
Mémoire			
Défauts de cache/s	8,020		
Mégaoctets disponibles	4 225,000		
Pourcentage d'octets dédiés utilisés	47,653		

I.4.5. Gestion de la Mémoire



I.4.5. Gestion de la Mémoire

❖ Monoprogrammation

Exemple : MS DOS

- Un seul processus en mémoire à la fois.

En pratique :

- Partie de l'espace d'adressage réservée au système d'exploitation (ROM + SE chargé au démarrage)
- A la fin du programme, retour à l'interpréteur de commande qui demande le prochain programme à lancer.

I.4.5. Gestion de la Mémoire

❖ Multiprogrammation

- Facilite le développement de programmes en les fractionnant en processus indépendants.
- Elle permet une maximisation de l'utilisation des ressources processeur.
- Problème : Comment organiser la mémoire de façon la plus efficace possible.

I.4.5. Gestion de la Mémoire

❖ Multiprogrammation avec partitions fixes

- Division de la mémoire en partitions (si possible inégales).
- Quand une tâche arrive, elle est placée dans une file d'attente :
 - une file d'attente par partition de mémoire
 - une seule file d'attente pour toute les partitions
- Réallocation
- protection

I.4.5. Gestion de la Mémoire

❖ Va et vient ou swapping

Mémoire insuffisante pour contenir tous les processus courant:

- Lorsqu'un processus demande l'allocation d'une page de mémoire et qu'il n'y en a pas de disponible en mémoire centrale, le noyau traite un défaut de page.
- Il choisit une page (qui n'a pas été utilisé depuis long-temps) et l'écrit sur une partition spéciale du disque dur.
- La place libérée est alors attribuée au processus demandeur.

Nécessité de placer certains de ces processus sur le disque:

- Ce mécanisme demande la réservation d'une (ou plusieurs) partition spéciale sur l'un des disques durs, nommée **partition de swap**.

Ramener les processus sur le disque en mémoire:

- Il faudra donc ramener régulièrement des processus sur le disque en mémoire centrale et inversement.
- C'est ce qu'on appelle le **va-et-vient** ou **swapping**.

I.4.5. Gestion de la Mémoire

❖ Va et vient ou swapping

- La mémoire disponible pour les processus est donnée par la somme de la taille de mémoire physique (RAM) et des partitions de swap.
- Bien entendu, les performances du système se dégradent lorsque la fréquence des défauts de page augmente ; dans ce cas, il faut augmenter la mémoire physique.
- Sur un système typique, la partition de swap est deux à trois fois plus grande que la mémoire centrale. **Exemple** : PC avec **32Mo** de **RAM**, partition de **swap** de **64Mo**
- Le système **UNIX** fonctionne en mémoire **virtuelle paginée**. Ceci permet de faire fonctionner des processus demandant une quantité d'espace mémoire supérieure à la mémoire physique installée.

I.4.5. Gestion de la Mémoire

❖ Technique de la pagination

- A l'intérieur des processeurs actuels, il y a une **unité de gestion de la mémoire (MMU)** qui fait correspondre des adresses virtuels à des adresses physiques.
- Un programme travaille sur un espace d'adressage virtuel.
- Cet espace est divisé en unités appelées **pages** (**512 à 4096** octets) et les unités correspondantes en mémoire physique sont appelées **cadres de pages** (page frames).
- La **MMU** dispose d'une table d'indirection des pages pour convertir les adresses virtuelles en adresses physiques.
- Les pages virtuelles qui ne sont pas mappées en mémoire centrale sont identifiées grâce à un bit de présence/absence.
- Les transferts RAM/disque se font par pages entières.

I.4.5. Gestion de la Mémoire

❖ Technique de la pagination

Pagination = Espace d'adressage virtuel plus grand que la mémoire physique.

Si un processus essaie de faire appel à une page non présente en mémoire physique :

- Déroulement du processeur (**défaut de page**) pour rendre la main au SE.
- Le SE sélectionne un cadre de page peu utilisé et sauve son contenu sur le disque.
- Transfère la page demandée dans le cadre de page libéré.
- Modifie la correspondance.
- Recommence l'instruction déroutée.

Gestion par un algorithme de remplacement de pages.

I.4.6. Les Entrées/Sorties (E/S)

Le SE a la tâche importante de contrôler les périphériques d'Entrées/Sorties (E/S)

❖ Fonctions :

- Emission des commandes vers les périphériques.
- Interception des interruptions.
- Gestion des erreurs.

❖ But :

- Fournir une interface simple entre les périphériques et le système.
- Interface identique pour tous les périphériques..

I.4.6. Les Entrées/Sorties (E/S)

❖ Deux catégories :

- **Périphériques par bloc** : informations stockées par blocs de taille fixe, chacun possédant sa propre adresse. (ex :disque).
- **Périphériques par caractères** : information qui circule sous la forme d'un flot de caractères, sans aucune structure de bloc. (ex : clavier, écran, imprimante, souris).

❖ Deux parties dans une unité :

- un composant mécanique, le périphérique (ex : disque).
- un composant électronique, le **contrôleur de périphérique** (ex : contrôleur IDE).

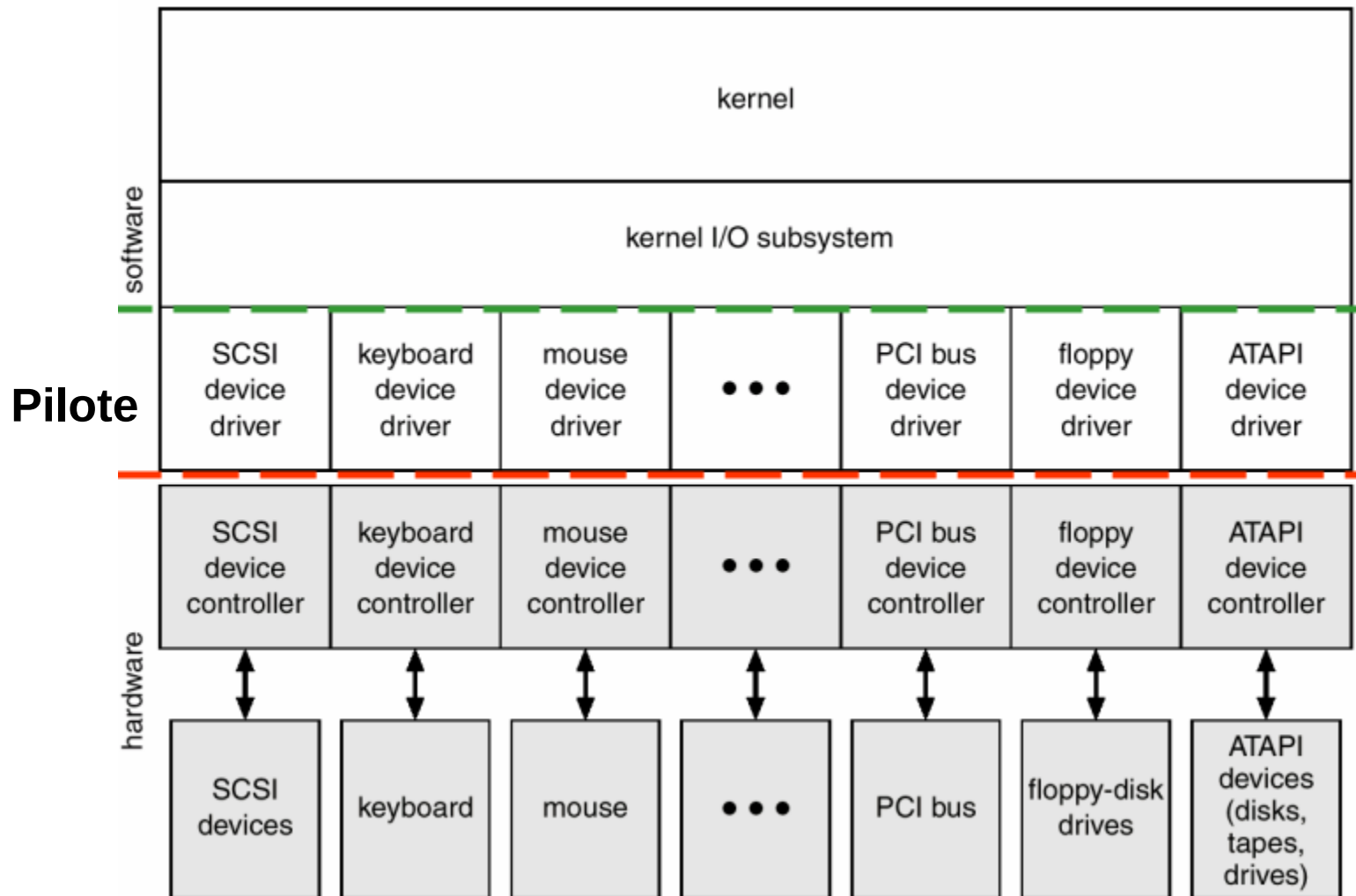
I.4.6. Les Entrées/Sorties (E/S):

❖ La Communication :

- Interface entre contrôleur et périphérique de très bas niveau.
- Le **contrôleur** possède des **registres** qui permettent la **communication** avec le **processeur**.
 - **Ecriture dans ces registres** : le SE ordonne au périphérique de délivrer des données, d'en accepter ou d'effectuer une action donnée.
 - **Lecture** : le SE peut connaître l'état du périphérique, savoir s'il est capable d'accepter une nouvelle commande.
- Certains périphériques sont équipés d'un tampon de données que le SE peut lire ou écrire.

I.4.6. Les Entrées/Sorties (E/S):

❖ Structure E/S d'un noyau (driver=pilote)



I.4.6. Les Entrées/Sorties (E/S):

❖ Les E/S mappées en mémoire :

Le processeur communique avec les registres de contrôle et les tampons de données, deux cas possibles :

- un numero de **port d'E/S** est assigné à chacun des registres de contrôle et il existe des instructions spéciales pour lire et/ écrire sur ces ports.
- **E/S mappées en mémoire**, chaque registre de contrôle se voit attribuer une adresse mémoire unique à laquelle aucune mémoire n'est assignée.

I.4.6. Les Entrées/Sorties (E/S):

❖ Les E/S mappées en mémoire :

Dans les ordinateurs de bureau actuels:

- les **E/S** sont mappées en **mémoire**,
- les adresses entre **640ko** et **1Mo** sont par exemple marquées comme étant destinées au bus **PCI** et non à la mémoire.

La puce de pontage **PCI** filtre les **adresses**.

I.5. Types d'Architecture des SE

I.5.1 Structure d'un SE

Le **Noyau**:

- C'est le programme qui assure la gestion de la mémoire, le partage du processeur entre les différentes tâches à exécuter et les entrées/sorties de bas niveau.
- Il est lancé au démarrage du système (le boot) et s'exécute jusqu'à son arrêt.
- C'est un programme relativement petit, qui est chargé en mémoire principale.

Le **rôle** principal du noyau

- Assurer une bonne répartition des ressources de l'ordinateur (mémoire, processeur(s), espace disque, imprimante(s), accès réseaux) sans intervention des utilisateurs.
- Il s'exécute en mode superviseur, c'est à dire qu'il a accès à toutes les fonctionnalités de la machine : accès à toute la mémoire, et à tous les disques connectés, manipulations des interruptions, etc.

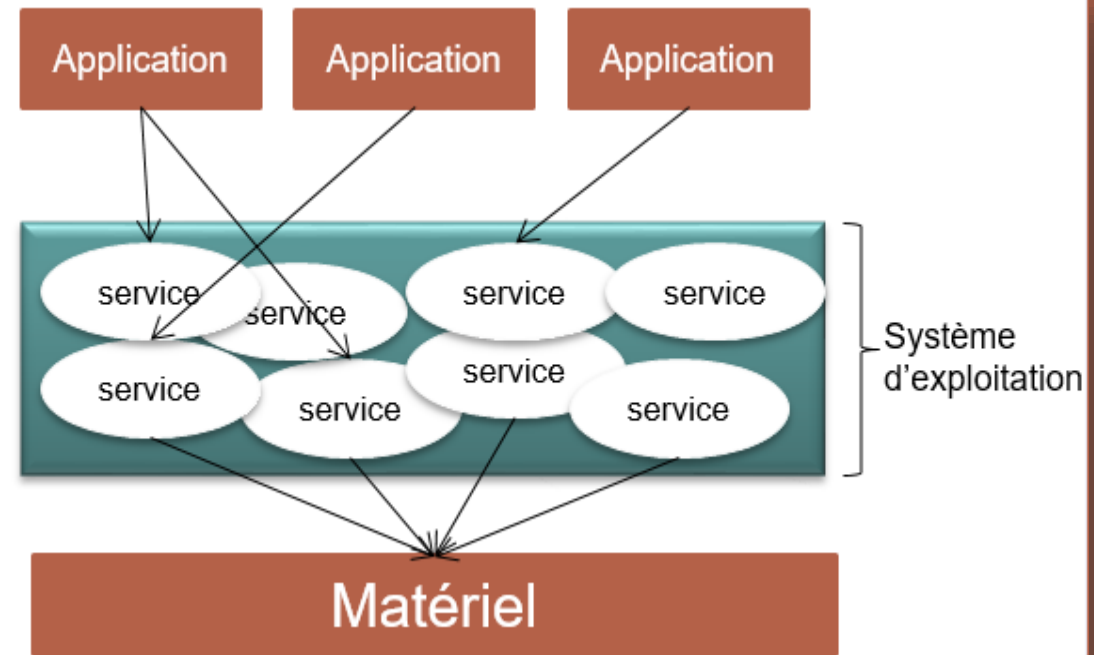
I.5.1. Structure d'un SE

Tous les **autres programmes** qui s'exécutent sur la machine fonctionnent en **mode utilisateur** :

- ils leur est interdit d'accéder **directement** au matériel et d'utiliser certaines instructions.
- Chaque **programme utilisateur** n'a ainsi accès qu'à une certaine partie de la mémoire principale, et il lui est impossible de lire ou écrire les zones mémoires attribuées aux autres programmes.
- Lorsque l'un de ces programmes désire accéder à une ressource gérée par le noyau, par exemple pour effectuer une opération d'entrée/sortie, il exécute un **appel système** . Le **noyau** exécute alors la fonction correspondante, après avoir **vérifié** que le programme appelant est **autorisé** à la réaliser.

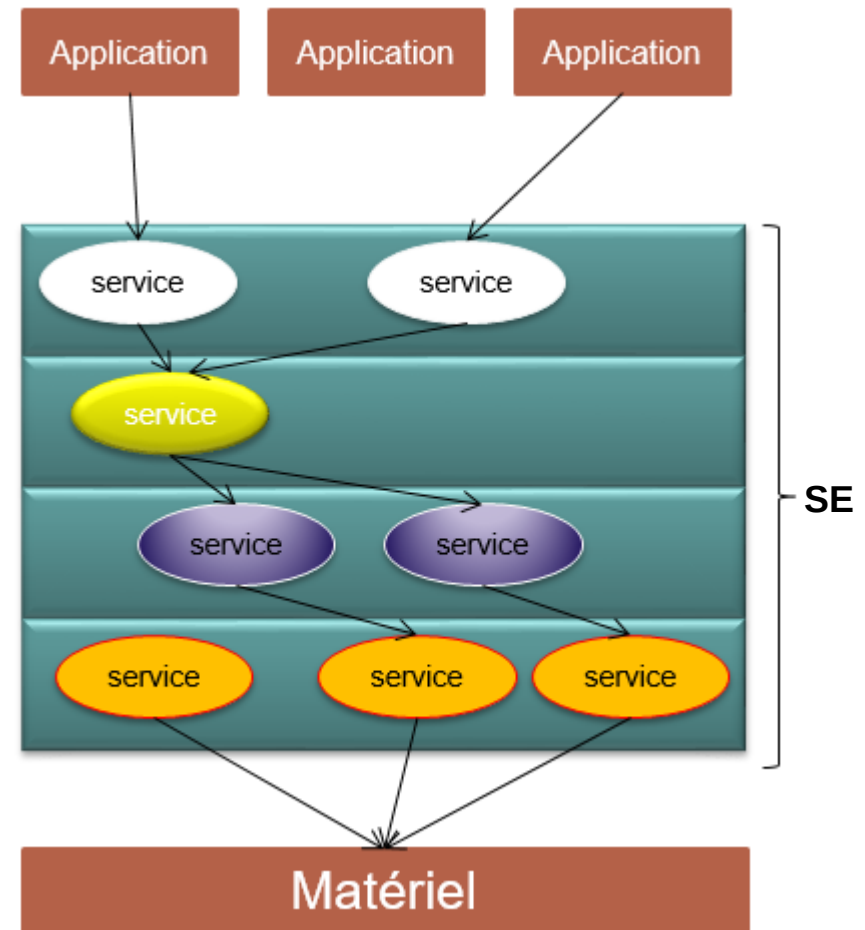
I.5.2. Système Monolithiques

- Un seul bloc contenant l'ensemble des services système (mode noyau).
- Facilité de conception
- Performance peut être au RDV ...
- Code dur à maintenir
- Usine à gaz !!
- Dos, très vieux UNIX et Linux, etc.



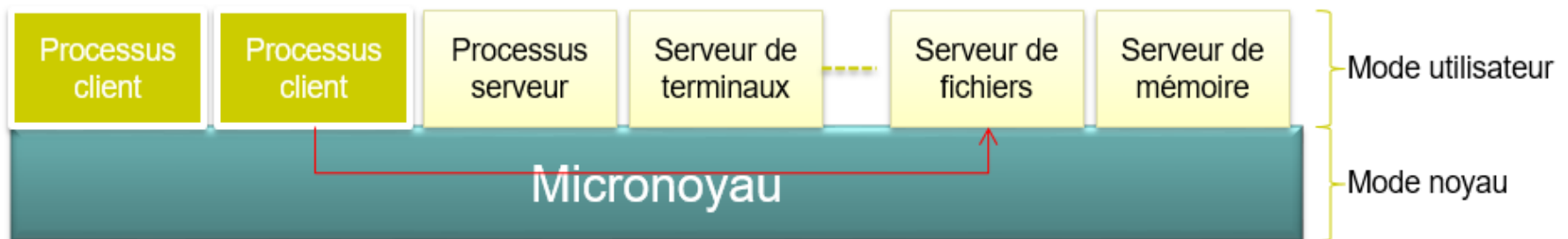
I.5.3. Système Multicouches

- OS organisé en hiérarchie de couches. Chacune construite sur la base des services offerts par la couche inférieure.
- Facilité de conception et de développement
- Code plus organisé et maintenable.
- Chargement des fonctionnalités à la demande
- Linux, BSD, SOLARIS



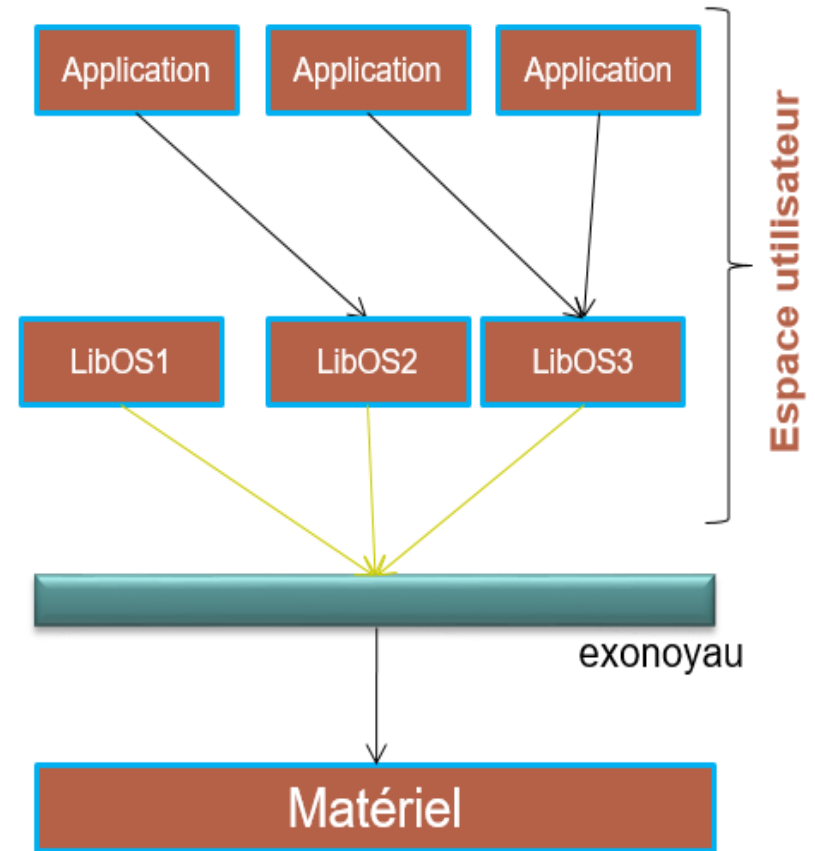
I.5.4. Système Micronoyau

- Déplace plusieurs fonctions de l'OS vers des « **processus serveur** » s'exécutant en mode utilisateur → réduction au maximum de la taille du code privilégié (en mode noyau).
- **But:** gérer les communications entre applications et serveurs pour:
 - Renforcer la politique de sécurité
 - Permettre l'exécution de fonctions système (accès aux registres d'E/S, etc.).
- Fiabilité augmentée: si un processus serveur « crash », le système continue à fonctionner et il est possible de relancer ce service sans redémarrer.
- Modèle facilement étendu à des systèmes distribués
- MAC OS X, GNU HURD, WindowsNT



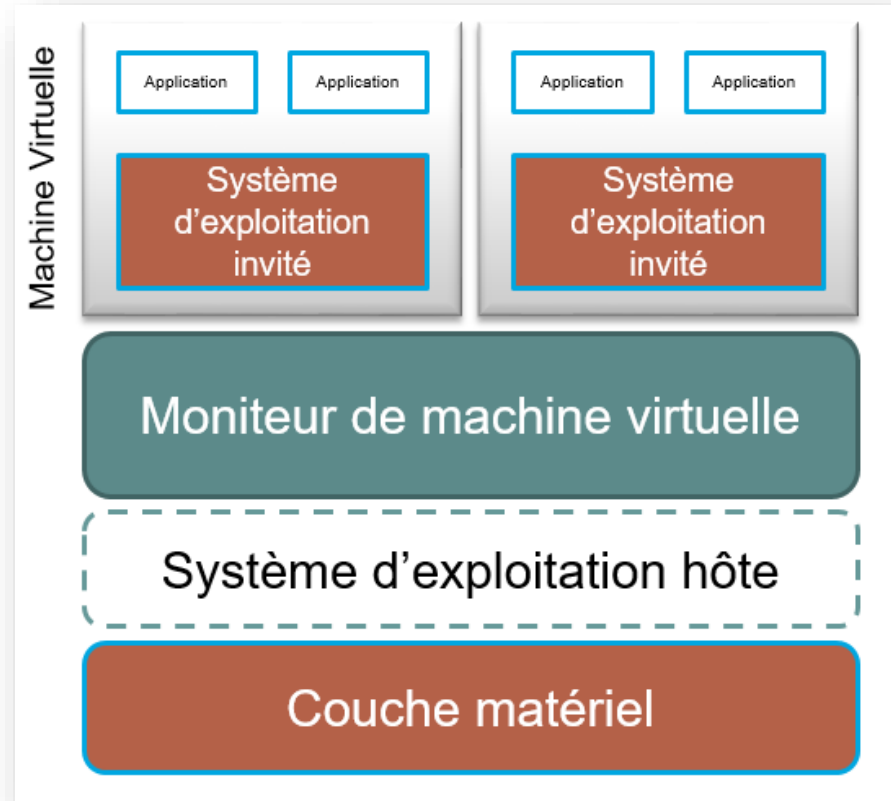
I.5.5. Système Exonoyau

- Le noyau se contente de **multiplexer** et **protéger** l'accès aux ressources.
- Tout le traitement qu'on trouve habituellement dans un noyau (mémoire virtuelle, système de fichiers, ...) est délégué à l'espace utilisateur via des bibliothèques appelées «LibOS».
- Plusieurs LibOS peuvent être utilisées en parallèle.
- Gain de performance
- Pb de sécurité
- Isolation de bugs
- Domaine encore balbutiant.



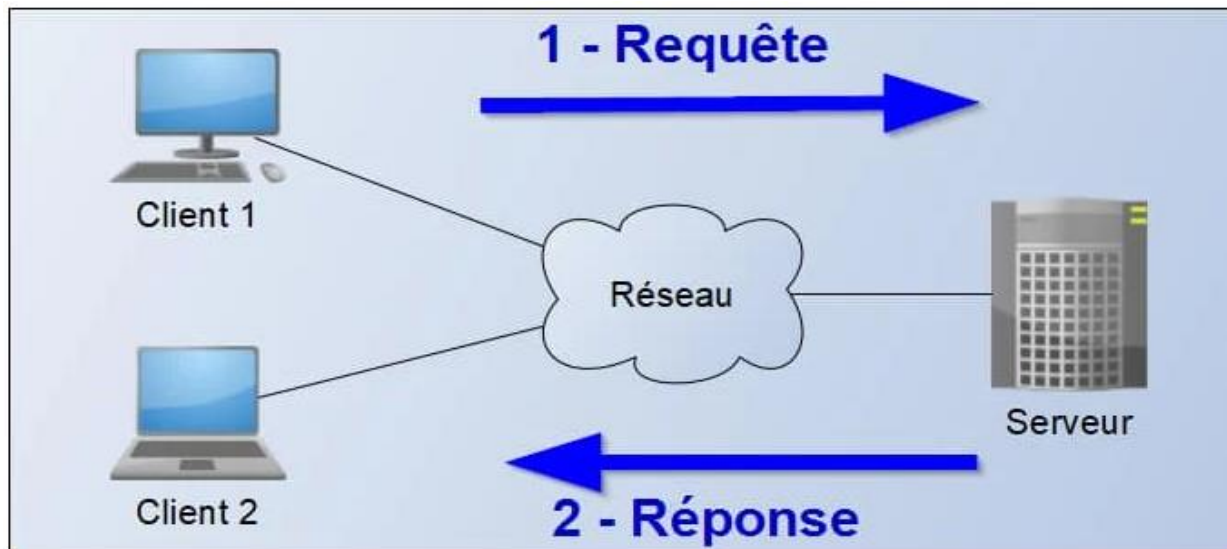
I.5.6. Les Machines Mirtuelles

- Possibilité de mettre plusieurs OS sur une machine physique.
- Le **moniteur de machine virtuelle** (hyperviseur) intercepte les instructions privilégiées envoyées par l'OS invité, les vérifie (politique de sécurité) et les exécute.
- Chaque machine virtuelle peut exécuter son **propre SE**.
- **XEN, VMWare, QEMU, VirtualBox**, etc



I.5.7. Architecture client/serveur

- C'est un modèle fondamental dans le domaine informatique, permettant une **interaction** efficace entre les **machines connectés en réseau**.
- Elle offre une structure claire pour la communication et la gestion des ressources, mais elle peut présenter des limitations face à des charges importantes.



I.5.7. Architecture client/serveur

- La plupart des fonctionnalités d'un SE sont reportées dans des processus utilisateurs.
- Pour demander un service comme la lecture d'un bloc de fichier, le **processus utilisateur** ou **processus client** envoie une **requête** à un **processus serveur** qui effectue le travail et envoie une **réponse**.
- Le **noyau** ne gère que la communication entre les **clients** et les **serveurs**.
- Cependant, le **noyau** est souvent obligé de gérer certains **processus serveurs critiques** comme les **pilotes** de périphériques qui adressent directement le matériel.
- En outre, ce modèle est bien adapté aux **systèmes distribués**. Un client n'a pas besoin de savoir si le SE fait exécuter sa requête par un serveur de sa propre machine ou celui d'une machine distante.

I.5.8. Autres Types

- **OS temps partagé**: garantir le partage équitable du temps processeur et des ressources dans le but de **maximiser le temps de traitement** et de **réduire le temps de réponse moyen**.
- **OS temps réel**: **garantir les temps de réponse**
 - **Systèmes à contraintes souples/molles**: systèmes acceptant des variations minimales de temps de réponse (systèmes multimédias)
 - **Systèmes à contraintes dures**: gestion stricte du temps pour conserver l'intégrité du système (déterminisme logique et temporel et fiabilité)
- **OS embarqué**: OS prévus pour fonctionner sur des machines de **petite taille**, (PDA ou des appareils électroniques autonomes: sondes spatiales, robot, ordinateur de bord, etc.), possédant une **autonomie réduite**

Travaux Pratiques: Commandes Windows

Executer la commande **DRIVERQUERY** et décrire la sortie

Afficher le moniteur de ressources, la capture ci-dessous: **perfmon**

Afficher la clé de licence de votre SE Windows: **wmic path softwarelicensing service get OA3xOriginalProductKey**

